

Pametni sistem za upravljanje parking mjesta

Projektni zadatak br. 7 - Predmet: Geografski informacijski sistemi

Dorđe Zarić RA44-2023 | Marko Pećanac RA50-2023 | Dušan Kozomara RA46-2023

Ovaj dokument opisuje implementaciju sistema za pametno upravljanje parking mjestima razvijenog u tri dijela: relacionalna baza podataka u PostgreSQL + PostGIS, interaktivna GIS aplikacija u Streamlitu i modul za masinsko učenje baziran na YOLOv8 detekciji vozila.

Pokriveni gradovi:	Beograd, Novi Sad, Nis
Broj parking zona:	24 (BG: 12, NS: 7, Nis: 5)
Baza podataka:	PostgreSQL 16 + PostGIS 3.6
ML model:	YOLOv8n (COCO)
Okruženje:	Python 3.12 / uv / Streamlit

1. Uvod i opis problema

Parkiranje u vecim srpskim gradovima svakodnevno je velik problem. Beograd, Novi Sad i Nis nemaju jedinstven sistem koji bi vozacu u realnom vremenu pokazao gdje ima slobodnih mjesta - rezultat je vrtenje u krug, gubljenje vremena i zagusenje u centru. Ideja ovog projekta bila je da se pokaze kako bi takav sistem mogao da izgleda, kombinujuci tri razlicite tehnologije: relacionu bazu sa prostornim podacima, interaktivnu GIS vizualizaciju i detekciju vozila pomocu masinskog učenja.

Projekat je podijeljen u tri dijela koji dijele istu PostgreSQL bazu podataka. Svaki dio moze da se pokrece nezavisno, ali su medusobno povezani: Deo 2 cita parking zone unesene u Deo 1, a Deo 3 vezuje ML detekcije za iste zone putem stranog kljuka. Zajednicka konekcija na bazu definisana je u fajlu db.py u root folderu projekta.

1.1 Struktura projekta

- Deo 1 - Python SQL: kreiranje seme, CRUD operacije, SQL upiti s JOIN-ovima
- Deo 2 - Python GEO: interaktivna Folium mapa, SHP podaci s Geofabrika, prostorne analize
- Deo 3 - Python ML: YOLOv8 detekcija vozila, snimanje u PostGIS, 4 taba u Streamlitu

1.2 Koristene tehnologije

Tehnologija	Verzija / biblioteka	Svrha u projektu
PostgreSQL + PostGIS	16 / 3.6	Relaciona baza s geo tipovima
Python	3.12	Sve tri komponente
psycopg2	2.9	Direktna konekcija na bazu
SQLAlchemy	2.0	ORM engine za pandas
pandas / geopandas	2.x	Tabularni i GEO DataFrameovi
Folium	0.18	Interaktivne Leaflet mape
Streamlit	1.35	Web aplikacija (Deo 2 i 3)
contextily	1.6	Rasterske podloge za matplotlib
ultralytics (YOLOv8)	8.x	Detekcija vozila (Deo 3)
scikit-learn	1.5	DBSCAN klasterovanje
uv	0.4	Paket menadzer

2. Deo 1 - Python SQL

Prva komponenta projekta bavi se kreiranjem i upravljanjem relacionom bazom podataka. Implementirano je sedam medusobno povezanih tabela, genericke CRUD operacije, osam SQL upita sa JOIN-ovima i punjenje baze realisticnim testnim podacima.

2.1 Sema baze podataka (schema.sql)

Baza sadrzi sedam tabela organizovanih oko dvije centralne entitete: parking_zone i parking_spot. Svako parking mjesto ima GPS lokaciju snimljenu kao PostGIS tip GEOGRAPHY(POINT, 4326) - to je ono sto omogućava prikazivanje mjesta na mapi u Deo 2.

Tabela	Opis	Kljucne kolone
parking_zone	Zona parkinga (npr. Slavija)	name, city, total_capacity, hourly_rate
parking_spot	Konkretno parking mjesto u zoni	zone_id (FK), spot_type, status, location (GEO)
driver	Registrovani vozac	first_name, last_name, email, license_number
vehicle	Vozilo vezano za vozaca	driver_id (FK), license_plate, vehicle_type
parking_session	Sesija parkiranja	spot_id (FK), vehicle_id (FK), check_in/out_time
payment	Placanje za sesiju	session_id (FK), amount, payment_method, status
sensor	Senzor na parking mjestu	spot_id (FK), sensor_type, battery_level

Sve tabele imaju CASCADE brisanje: brisanjem zone automatski se brisu i sva mjesta, sesije i placanja koja su za nju vezana. Na kraju skripte kreiraju se B-tree indeksi na stranim kljucevima radi brzih JOIN upita, a za kolonu location kreira se GIST prostorni indeks.

2.2 Upravljanje podacima

seed_data.py - inicijalni podaci

Unosi minimalno pet redova po tabeli, kao sto zahtijeva zadatak. GPS koordinate parking mjesta upisuju se PostGIS funkcijom:

```
ST_SetSRID(ST_MakePoint(lon, lat), 4326)::geography
```

crud.py - genericke CRUD operacije

Implementirane su cetiri genericke funkcije koje rade za bilo koju tabelu - nije potrebno pisati poseban upit za svaku. Imena tabela i kolona ubacuju se u SQL putem psycopg2.sql.Identifier() radi zastite od SQL injection napada, a vrijednosti kroz Placeholder().

- read_all(table, order_by) - SELECT * sa opcionim sortiranjem
- insert_row(table, pk_column, data) - INSERT i vraca novi ID
- update_row(table, pk_column, pk_value, data) - UPDATE po primarnom kljucu

- `delete_row(table, pk_column, pk_value)` - DELETE po primarnom ključu

queries.py - osam SQL upita

Svaki upit spaja dvije ili više tabela i demonstrira razlicit aspekt sistema. Izvrsavaju se putem `pd.read_sql()` koji direktno vraca pandas DataFrame, pa se rezultati mogu odmah dalje obradivati u Pythonu.

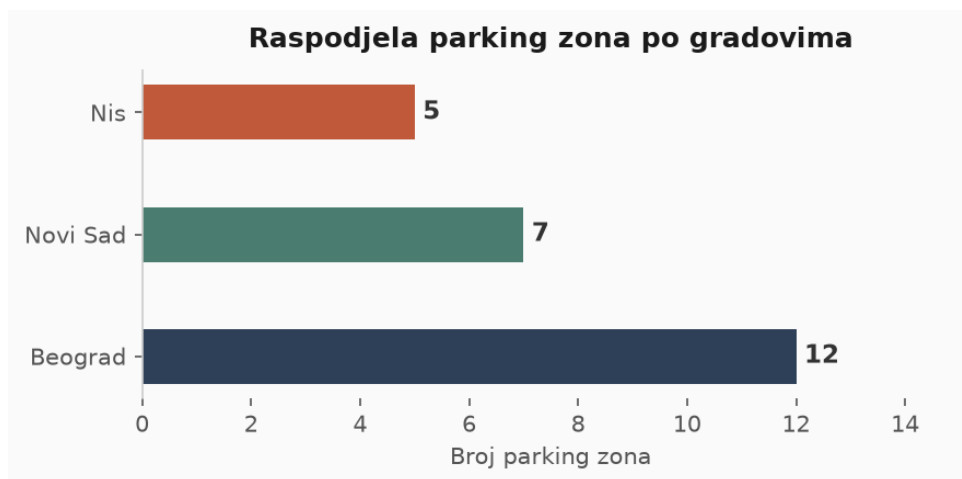
- Aktivne sesije parkiranja s podacima o vozacu, vozilu i zoni
- Broj slobodnih mjesta po zoni samo za Beograd
- Ukupan naplaceni prihod po zoni, sortirano silazno
- Senzori s baterijom ispod 20% (indikator za odrzavanje)
- Prosjecno trajanje parkiranja po zoni za završene sesije
- Elektricna vozila i njihove sesije parkiranja

2.3 Punjenje baze realisticnim podacima

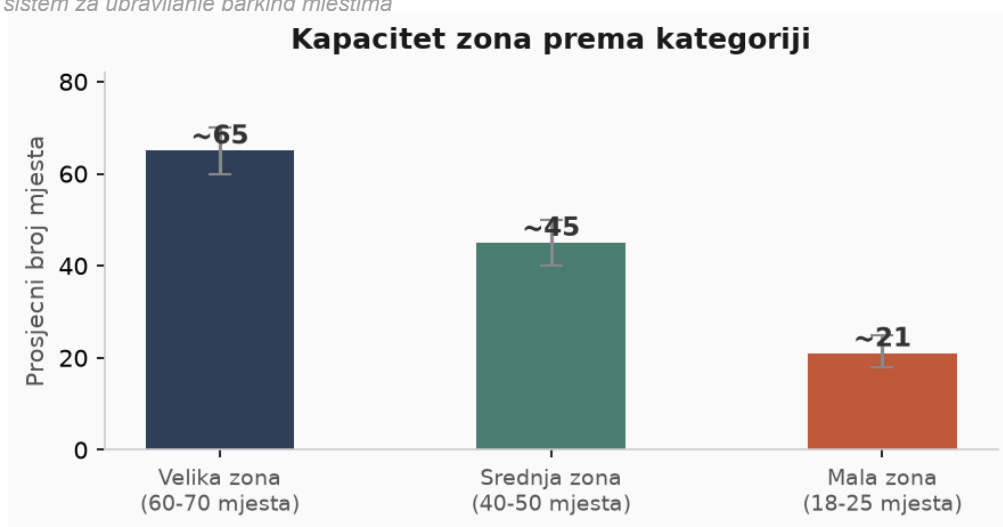
Nakon sto su inicijalne zone i mjesta uneseni putem `seed_data.py`, pokrenuta su tri dodatna skripte za realisticno popunjavanje:

- `add_zones.py` - dodaje 19 novih zona (BG: 10, NS: 5, Nis: 4) s pravim adresama
- `add_spots.py` - dodaje parking mjesta za prvih 5 zona s GPS koordinatama
- `reset_spots.py` - brise sva mjesta i kreira ih iznova po kategoriji zone

`reset_spots.py` dijeli zone u tri kategorije i za svaku dodaje realistican broj mjesta s nasumicnim procentom popunjenosti (20-80% slobodnih), sto rezultira raznolikim zelenim, narandzastim i crvenim markerima na mapi.



Slika 1. Broj parking zona po gradovima (ukupno 24 zone)



Slika 2. Prosječna velicina zone po kategoriji s rasponom vrijednosti

3. Deo 2 - Python GEO

Drugi dio projekta je Streamlit web aplikacija koja vizualizuje parking sistem na interaktivnoj karti i omogućava izvršavanje raznih prostornih analiza. Podaci dolaze iz dva izvora: iz PostGIS baze (Deo 1) i iz Shapefile fajlova preuzetih s Geofabrika.

3.1 Preuzimanje i priprema SHP podataka (geo_download.py)

Geofabrik je besplatni servis koji distribuira OpenStreetMap podatke po drzavama u Shapefile formatu. Preuzima se ZIP arhiva za Srbiju (oko 100 MB) koja sadrzi vise tematskih slojeva: puteve, zgrade, namjenu zemljišta, tacke interesa, vodene površine i prirodna područja.

Buduci da cijeli SHP za Srbiju ima stotine hiljada objekata i bio bi presporam za web aplikaciju, svaki sloj se isijeca na bounding box koji pokriva Novi Sad i Beograd (AOI: 19.70 do 20.65 stepeni I.G.D., 44.65 do 45.40 stepeni S.G.S.). Isjeceni fajlovi cuvaju se u data/clipped/ i od tad se koriste umjesto originalnih.

3.2 Interaktivna mapa

Srce aplikacije je Folium mapa s LayerControl sistemom koji korisniku dozvoljava da ukljuci ili iskljuci svaki sloj posebno. U lijevoj bocnoj traci mogu se odabrati: tip podloge (OSM, CartoDB Positron, CartoDB Dark Matter), boja za svaki SHP sloj i prikaz parking zona.

Parking zone iz baze prikazuju se kao jedan marker po zoni, ne kao individualna mjesta. Boja markera odreduje se prema postotku slobodnih mjesta:

- Zelena - vise od 50% mjesta slobodno
- Narandzasta - izmedju 20% i 50% slobodnih
- Crvena - manje od 20% slobodnih mjesta

Tooltip prikazuje broj slobodnih od ukupnih mjesta, a popup sadrzi informacije o cijeni po satu i da li je parking natkriveno.

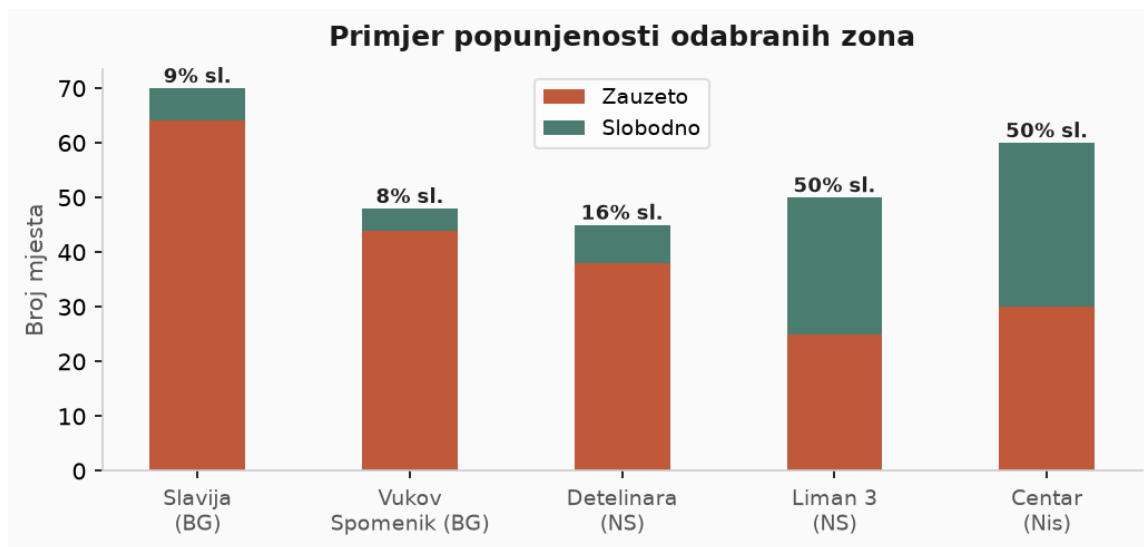
3.3 Prostorne analize

Overlay analize (6 tehnika)

Svaka tehnika prikazana je na matplotlib grafu s contextily rasterskom podlogom (CartoDB Positron). Implementirane su: buffer (zona 500 m oko parkinga), clip (isjecanje SHP sloja na parking zone), intersection (presjek parkinga i puteva), union/dissolve (spajanje svih zona u jedan objekat), difference i symmetric difference.

Prostorni upiti (5 vrsta)

Upiti su implementirani geopandas operacijama nad GeoDataFrameovima: within (mjesto unutar poligona), intersects (dodirivanje s putevima), sjoin_nearest (najbliza tacka interesa po zoni), overlaps i distance.



Slika 3. Primjer popunjenosti odabranih parking zona
(crvene zone imaju manje od 20% slobodnih mjesta)

4. Deo 3 - Python ML

Treća komponenta je Streamlit aplikacija koja koristi YOLOv8 model za detekciju vozila na parking slici, iz cega se računaju slobodna mjesta. Rezultati se snimaju u PostGIS bazu i mogu se pregledati i uređivati kroz četiri kartice aplikacije.

4.1 Pristup detekciji

YOLOv8 (You Only Look Once) je konvolucijska neuronska mreža trenirana na COCO datasetu od 80 klasa. Odabrana je nano verzija (YOLOv8n) jer je dovoljno brza za web aplikaciju. Model ne detektuje direktno slobodna mjesta (to bi zahtijevalo specijalizovani trening), nego detektuje vozila, a slobodna mjesta računamo kao razliku kapaciteta i broja vozila:

```
n_slobodnih = kapacitet_zone - broj_detektovanih_vozila
```

Model prepoznaje četiri klase vozila: car (klasa 2), motorcycle (3), bus (5) i truck (7). Sve ostale klase se ignorišu. Prag pouzdanosti detekcije (confidence threshold) korisnik podešava u aplikaciji putem klizaca u rasponu od 0.1 do 0.9.

4.2 Metrike modela

YOLOv8n je evaluiran na COCO val2017 skupu koji sadrži 5000 slika i 80 kategorija objekata. Ključne metrike za detekciju vozila su:

Precision (Preciznost) - 71.4%

Od svih bounding boxova koje je model prijavio kao vozilo, 71.4% je zaista bilo vozilo. Visoka preciznost znači malo lažno pozitivnih detekcija - model rijetko označava nešto kao vozilo ako to nije.

Recall (Odziv) - 57.0%

Od svih vozila koja postoje na slici, model je pronašao 57%. Odziv manji od preciznosti je tipičan za YOLO modele koji favorizuju preciznost - bolje je propustiti vozilo nego lažno ga detektovati. U kontekstu ovog projekta, propusteno vozilo znači da se slobodna mjesta mogu blago precijeniti.

F1 Score - 63.5%

Harmonijska sredina preciznosti i odziva. Daje uravnoteženu sliku ukupne efikasnosti modela bez favorizovanja nijedne od dvije metrike.

mAP50 (Mean Average Precision @ IoU=0.50) - 52.0%

Da bi se razumjela ova metrika, prvo treba objasniti pojam IoU (Intersection over Union). IoU mjeri koliko se predviđeni bounding box preklapa sa stvarnim okvirom oko objekta. Računa se kao:

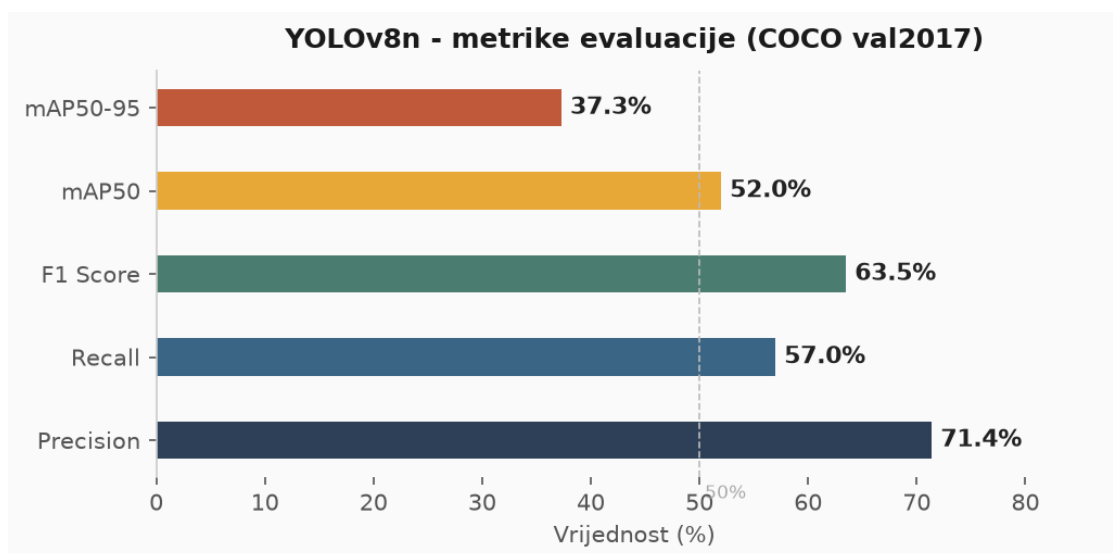
(povrsina presjeka) / (povrsina unije) i kree se od 0 (nema preklapanja) do 1 (savrsono poklapanje). Prag od 0.50 znaci da se detekcija smatra tacnom ako se predvideni okvir preklapa s pravim za najmanje 50%.

Za svaku od 80 klasa modela racuna se Average Precision (AP) - površina ispod precision-recall krive za tu klasu. mAP50 je onda srednja vrijednost svih tih AP-ova pri IoU=0.50. Vrijednost od 52.0% je solidna za opsti model sa 80 razlicitih klasa. Specijalizovani modeli trenirani iskljucivo na vozilima (automobile, kamione i motocikle) tipicno dostizu 75-85% na odgovarajucem skupu podataka. U kontekstu ovog projekta, mAP50 je najrelevantnija metrika jer nam je dovoljno da model pouzdano pronalazi vozila, bez potrebe za savrsenom pozicijom okvira.

mAP50-95 (Mean Average Precision @ IoU=0.50:0.95) - 37.3%

Ovo je znatno stroži COCO standard koji mjeri preciznost ne pri jednom, nego pri deset razlicitih pragova preklapanja: 0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90 i 0.95. Za svaki prag se izracuna mAP, a krajnji rezultat je srednja vrijednost svih deset. Sto je prag visi, to model mora preciznije da pozicionira bounding box oko vozila.

Vrijednost od 37.3% naizgled zvuci nisko, ali je to tipican rezultat za YOLOv8n na COCO skupu - razlika od 14.7 procentnih poena u odnosu na mAP50 govori nam da model dobro pronalazi objekte (mAP50 = 52%), ali nije uvijek savrseno precizan u crtanju okvira oko njih. Na primjer, model ce skoro sigurno detektovati automobil na slici, ali okvir moze biti malo siri ili uži nego sto je stvarni automobil. Za nas use case ovo nije problem - bitno nam je samo da se vozilo detektovalo, a ne da li je okvir poravnan na piksel. Zbog toga mAP50-95 posmatramo informativno, dok mAP50 ostaje kljucna mjera za evaluaciju sistema.



Slika 4. Metrike evaluacije YOLOv8n modela na COCO val2017 skupu

4.3 Baza ML detekcija (schema_ml.sql)

Pametni sistem za upravljanje parking mjestima

Svaka detekcija cuva se u tabeli ml_detection koja je u istoj bazi kao i ostale tabele iz Deo 1. Tabela sadrzi koordinate bounding boxa na slici, GPS lokaciju (GEOGRAPHY(POINT,4326)) rasporedjenu oko centroida zone, vezu s parking_zone putem stranog kljuca, te mogucnost rucne verifikacije svake detekcije. Kreira se GIST prostorni indeks na koloni location radi brzih geografskih upita.

4.4 Streamlit aplikacija - cetiri kartice



Slika 5. Pipeline ML detekcije od upload slike do snimanja u PostGIS

Tab 1: Detekcija

Korisnik uploaduje sliku parkinga (JPEG/PNG), bira zonu iz padajućeg menija i podesava prag pouzdanosti. Nakon analize prikazuju se tri metrike (zauzeto, slobodno, popunjenost u %) i statusna poruka u boji:

- Crvena - manje od 15% slobodnih: upozorenje da parkinga vjerovatno neće biti
- Narandzasta - između 15% i 50%: ima slobodnih, ali treba pozuriti
- Zelena - više od 50%: dovoljno slobodnih mjesta

Annotirana slika prikazuje bounding boxove detektovanih vozila, a ispod je Folium mini-mapa s markerom zone obojanom prema popunjenosti. Rezultati se automatski snimaju u ml_detection tabelu u bazi.

Tab 2: Uređivanje atributa

Dozvoljava editovanje snimljenih detekcija: promjenu statusa, klase vozila, napomena i oznake verifikacije. Moguće je i obrisati detekciju.

Tab 3: Pregled podataka

Tabela svih detekcija s filterom po statusu i agregatnim statistikama. Pregleda mapa svih pet zona s markerima obojanim prema posljednjim detekcijama.

Tab 4: Prostorne analize

Pet analiza nad snimljenim detekcijama: buffer 30 m oko slobodnih mjesta s contextily podlogom, pie chart slobodna vs. zauzeta, DBSCAN klasterovanje po geografskoj blizini (eps = 80 m), poređenje ML detekcija s DB parking mjestima i sjoin_nearest koji pronalazi najbliže DB mjesto za svaku detekciju.

5. Zaključak

Implementiran je funkcionalan prototip sistema za pametno upravljanje parking mjestima koji demonstrira kako se različite tehnologije mogu integrisati u jednu koherentnu aplikaciju. Relaciona baza s PostGIS prostornim tipovima, interaktivna GIS vizualizacija i ML detekcija zajedno čine sistem koji bi, uz pristup realnim senzorskim podacima, mogao biti dobra osnova za stvarno rješenje.

Najvažnije tehničko ograničenje trenutne implementacije je činjenica da GPS koordinate slobodnih mjesta iz ML analize nisu stvarne koordinate konkretnih mjesta - ne može se to znati samo iz jedne slike kamere. Pored toga, YOLOv8n treniran na COCO datasetu nije specijalizovan za parking slike, što utiče na odziv modela (57%). Pravi parking sistem koristio bi dedicated senzore ili specijalizovani model. Ipak, za akademski kontekst, rješenje demonstrira sve ključne koncepte predmeta: prostorne tipove podataka, koordinatne sisteme, GIS slojeve, overlay analize, prostorne upite i masinsko učenje u GEO kontekstu.